

CSE302 | Database Systems

Antu Chowdhury
Lecturer,
Dept. of CSE
East West University

DBMS Architecture

- A DBMS architecture defines how users interact with the database to read, write, or update information. A well-designed architecture and schema (a blueprint detailing tables, fields and relationships) ensure data consistency, improve performance and keep data secure.

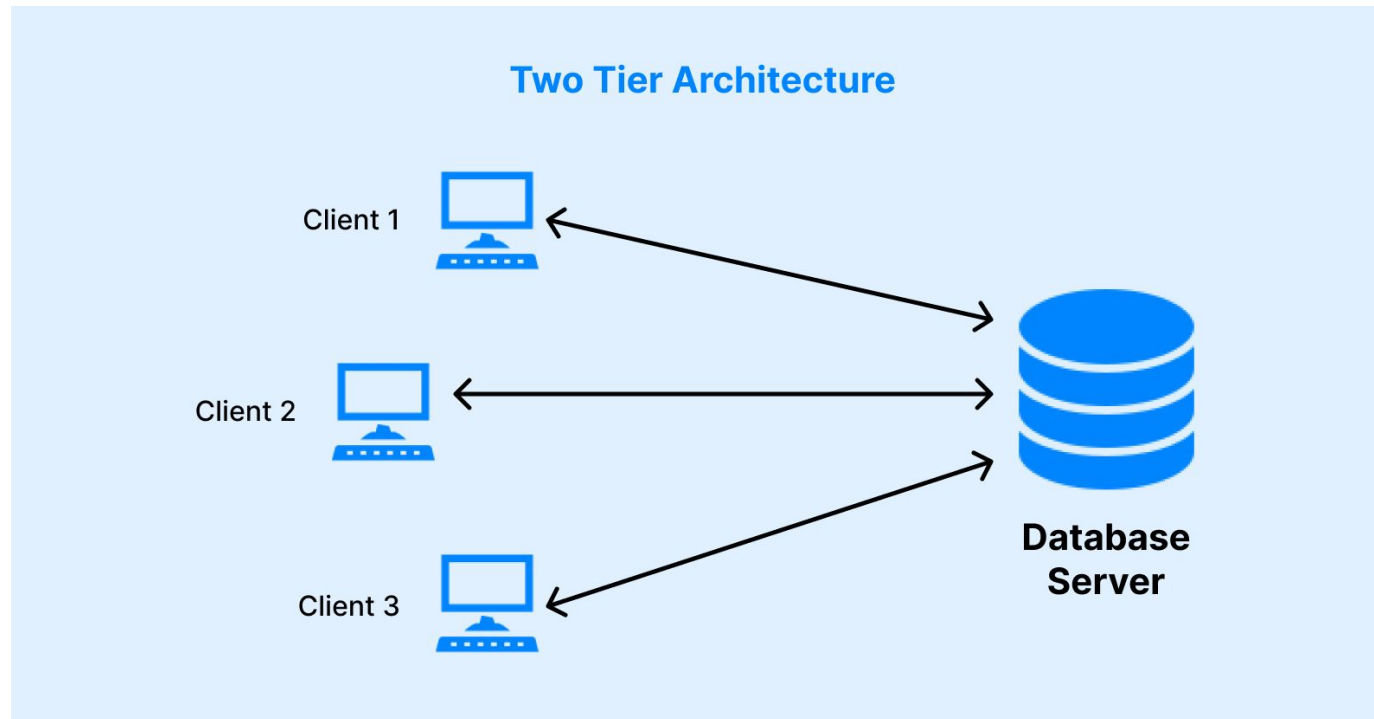
Types of DBMS Architecture

There are several types of DBMS Architecture that we use according to the usage requirements.

- 2-Tier Architecture
- 3-Tier Architecture

2-Tier Architecture

- The 2-tier architecture is similar to a basic client-server model. The application at the client end directly communicates with the database on the server side. APIs like ODBC and JDBC are used for this interaction. The server side is responsible for providing query processing and transaction management functionalities.



2-Tier Architecture Contd.

Applications:

- Going back to our *Railway Ticket Reservation* example, we usually use a web browser to book a ticket, in such a case our device acts as a client and the whole Railways database acts as the server which stores the information of every passenger. Another way of booking a ticket is by physically going to the Railway Station and filling the form, in such a case the ticket counter machine acts as a client that sends data to the Server.

- ***Major advantages of two-tier architecture :***

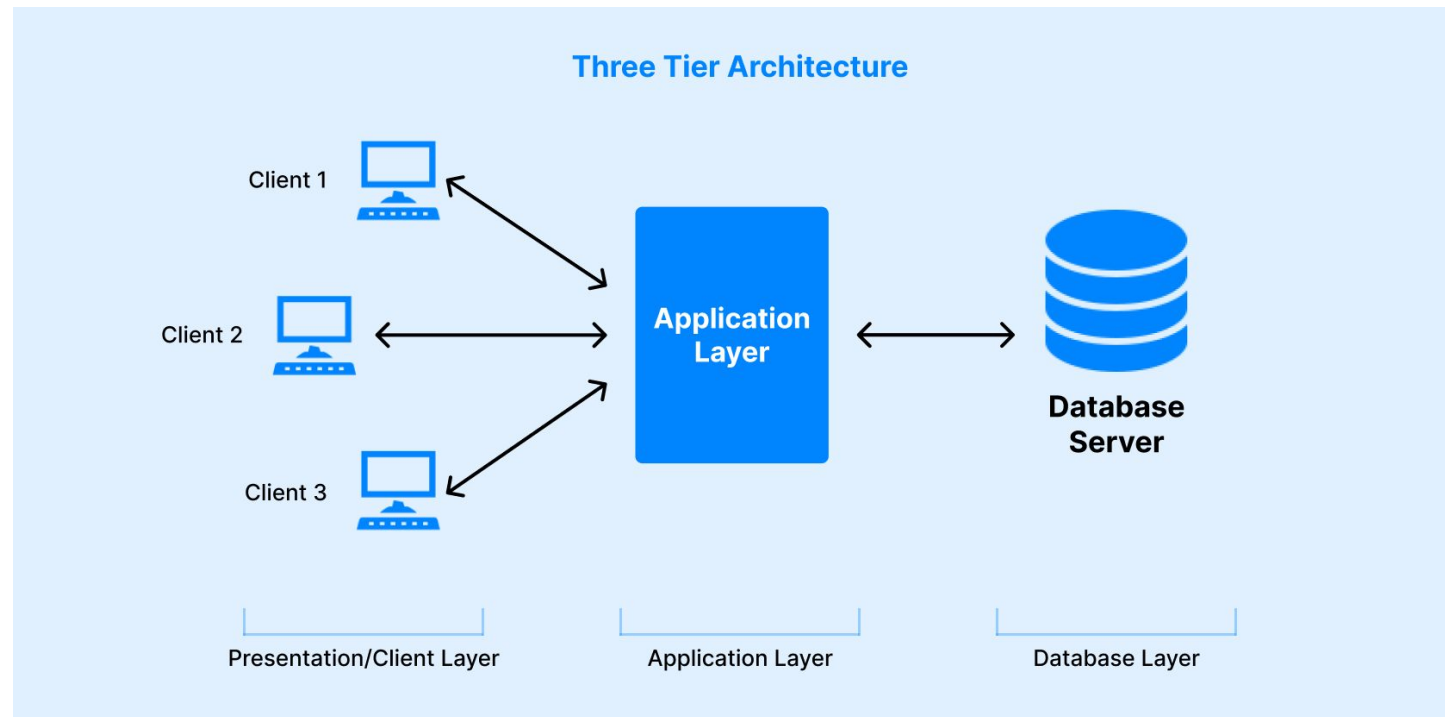
simplicity in design and maintenance, high performance.

Major disadvantages of two-tier architecture :

- Limited Scalability and Security issues.
- Not suitable for **large or web-based enterprise applications**

3-Tier Architecture

- In 3-Tier Architecture, there is another layer between the client and the server. The client does not directly communicate with the server. Instead, it interacts with an application server which further communicates with the database. This intermediate layer acts as a medium for the exchange of partially processed data between the server and the client. **This type of architecture is used in the case of large web applications.**



3-Tier Architecture Contd.

Presentation Layer (Client Layer)

- **Purpose:** It's the user interface that interacts directly with end users.
- **Functions:**
 - Display data to users.
 - Collect user inputs.
 - Send requests to the application server via HTTP or API calls.

Data Layer (Database Layer)

- **Purpose:** Responsible for **data storage, retrieval, and management.**
- **Functions:**
 - Stores all persistent data (e.g., user profiles, product catalogs, transaction records).
 - Executes queries received from the application layer.
 - Returns results securely and efficiently.

Application Layer (Business Logic Layer / Middleware)

- **Purpose:** Acts as a **bridge** between the presentation layer and the database layer.
- **Functions:**
 - Contains business logic (rules, validations, calculations).
 - Processes user requests and queries.
 - Communicates with the database layer to fetch or update data.
 - Ensures data consistency and security before sending back to the client.

3-Tier Architecture Contd.

Applications:

- **User Request:** A customer searches for “wireless headphones” on the website (Presentation Layer).
- **Business Logic:** The search request is sent to the Application Server, which processes it, applies filters, and checks availability (Application Layer).
- **Database Query:** The Application Layer queries the database for products matching “wireless headphones” (Data Layer).
- **Response:** The database returns results → the Application Layer formats them → the Presentation Layer displays them to the user.

Major advantages of three-tier architecture :

- Reduced load on the Query processor enhancing the Database Design and security.
- Scalability
- Security

Major disadvantages of two-tier architecture :

- Increased complexity, higher infrastructure and development costs

3-Tier Architecture Contd.

- **Example Technologies Used**

Layer	Common Technologies
Presentation	HTML, CSS, JavaScript, React
Application	Node.js, Django, Spring Boot
Database	MySQL, PostgreSQL, Oracle, MongoDB

Schema

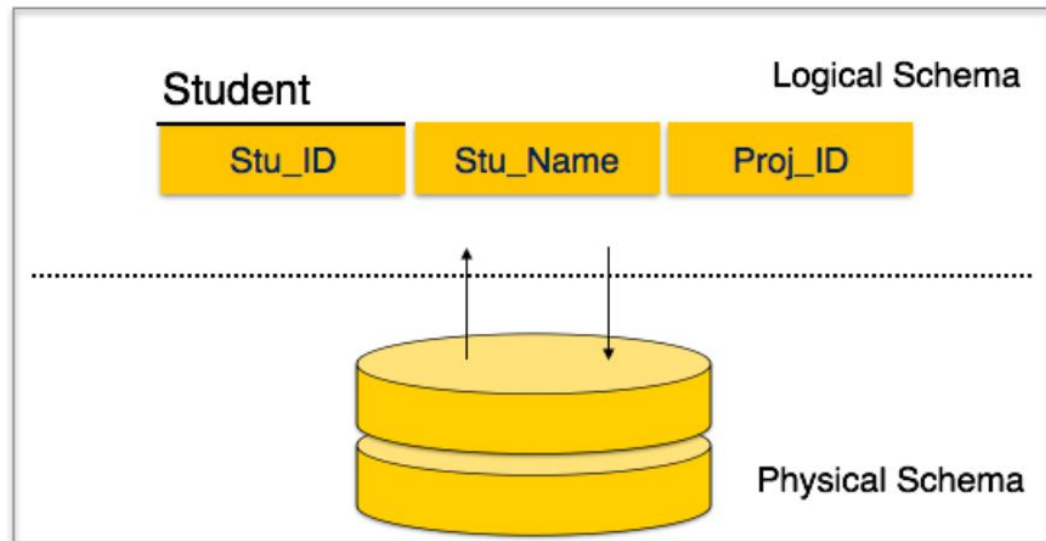
- *A database schema is the skeleton structure that represents the logical view of the entire database.*
- It defines how the data is organized and how the relations among them are associated.
-

Schema

- Student
 - roll no
 - reg no
 - email
 - name
 - Course
 - Course code or id
 - Course title
 - Credit
 - Teacher
 - Name
 - Email
 - Designation
- Logical representation of the DB
 - It can have a single table or a collection of tables

Schema

- A database schema can be divided broadly into two categories –

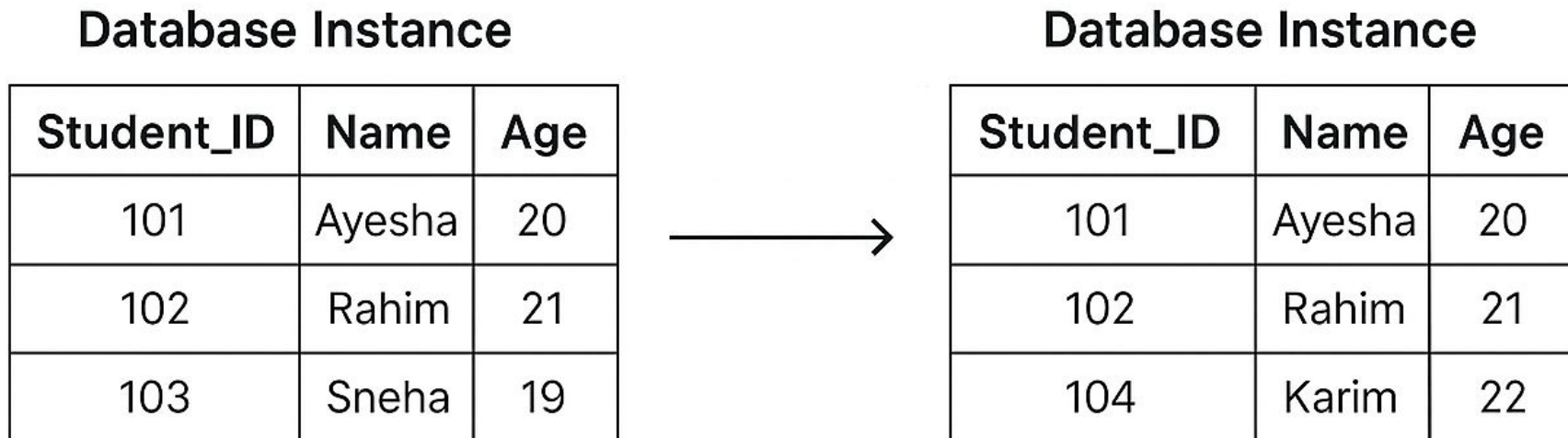


Physical Database Schema – This schema pertains to the actual storage of data and its form of storage like files, indices, etc. It defines how the data will be stored in a secondary storage.

Logical Database Schema – This schema defines all the logical constraints that need to be applied on the data stored. It defines tables, views, and integrity constraints.

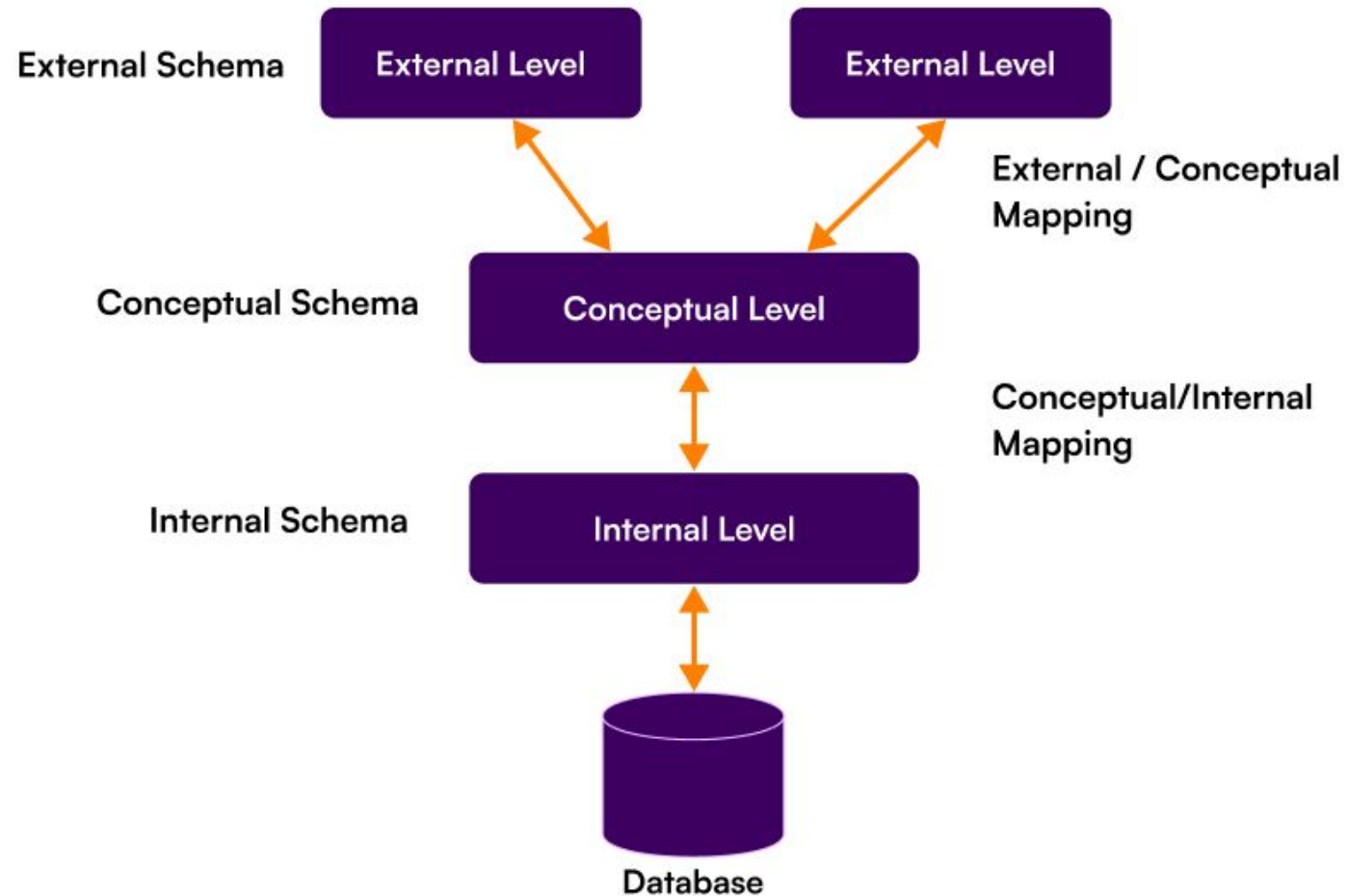
Database Instance

- A **database instance** refers to the **state or snapshot of a database** at a specific moment in time. It represents all the data stored in the database at that instant. As data changes due to insertions, deletions, or updates, the database instance also changes continuously.
- A **DBMS (Database Management System)** ensures that every database instance remains **valid and consistent** by enforcing all the defined **constraints, validations, and integrity rules** from the database schema.



3-Schema architecture

- The three schema architecture is also three-level architecture or Three Level of Abstraction
- This framework is used to describe the structure of a specific database system.
- The three schema architecture contains **three-levels**. It breaks the database down into three different categories.
 - Internal Schema (Physical Level)
 - Conceptual Schema (Logical Level)
 - External Schema (View Level)



Internal Schema (Physical Level)

- The **internal schema**, also known as the **physical level**, is the **lowest level of data abstraction** in a database. It describes **how data is actually stored** in the system — including details about **file organization, indexing, record placement, and storage structures** on the physical storage device (like hard drives or SSDs).

STORED_EMPLOYEE record length 60

Empno : 4 decimal offset 0 unique

Ename : String length 15 offset 4

Salary : 8,2 decimal offset 19

Deptno : 4 decimal offset 27

Post : string length 15 offset 31

Example: Consider a database storing information about employees. The Internal Schema would describe how employee data is stored in a specific file format or how the data is indexed for fast retrieval (e.g., B-trees, hash indexing).

Conceptual Schema (Logical Level)

- The **conceptual schema**, also known as the **logical level**, represents the **entire logical structure of the database** as seen by the database administrator. It defines **what data is stored**, the **relationships** among those data, and the **constraints** that govern them — but it **does not deal with how** the data is physically stored.

EMPLOYEE

Empno : Integer(4) Key

Ename : String(15)

Salary : String (8)

Deptno : Integer(4)

Post : String (15)

Example: In the case of the employee database, the Conceptual Schema would define entities like employees, departments, and projects, and how these entities are related (e.g., each employee belongs to a department).

External Schema (View Level)

- The External Schema defines how individual users or user groups view the data. It represents the user perspective and focuses on the specific requirements of users while hiding the rest of the database from that user group.

External
View

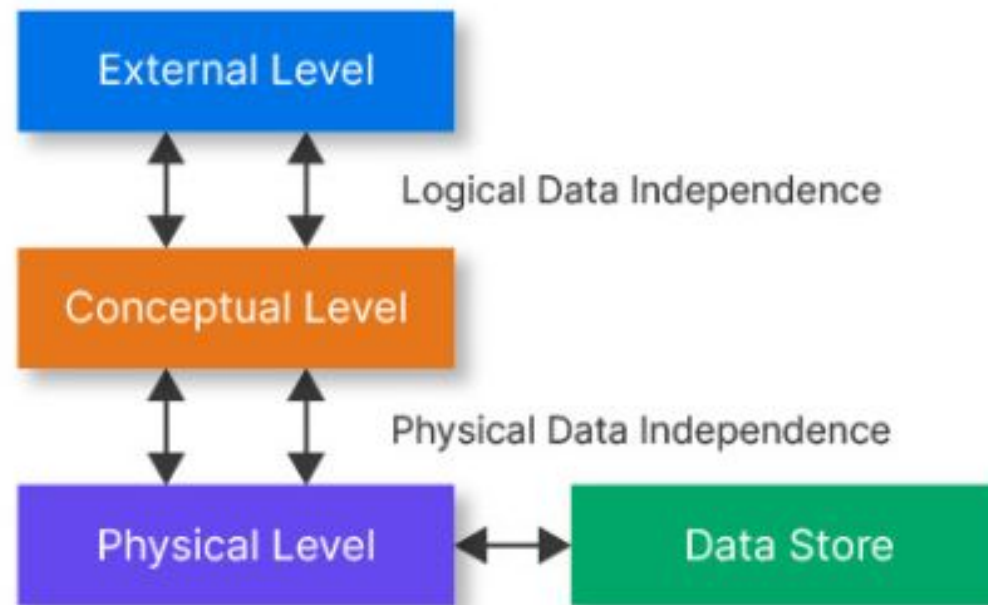
Empno	Ename
-------	-------

Empno	Ename	Salary	DeptNo
-------	-------	--------	--------

Example: For an employee database, one user may need to view the employees' personal information, while another may need to access employee performance data. Each of these user views is a part of the External Schema.

Data Independence

Data independence in DBMS refers to the capacity to change the schema (structure) of the database without affecting the application programs or user views that access the data. It is a fundamental concept that simplifies database maintenance and enhances transparency.



Data Independence

- **Physical Data Independence** can be defined as the ability to change the physical level without affecting the logical or Conceptual level. Physical data independence gives us the freedom to modify the - Storage device, File structure, location of the database, etc. without changing the definition of conceptual or view level.
- Example: For example, if we take the database of the banking system and we want to scale up the database by changing the storage size and also want to change the file structure, we can do it without affecting any functionality of logical schema.

Data Independence

- **Logical Data Independence** is a property of a database that can be used to change the logic behind the logical level without affecting the other layers of the database. Logical data independence is usually required for changing the conceptual schema without having to change the external schema or application programs. It allows us to make changes in a conceptual structure like adding, modifying, or deleting an attribute in the database.
- Example: If there is a database of a banking system and we want to add the details of a new customer or we want to update or delete the data of a customer at the logical level data will be changed but it will not affect the Physical level or structure of the database.

Thank You